MCP FIELD GUIDE / PYTHON

Build & Deploy Your Own MCP Server

From an empty folder to a secured, Dockerized server that any AI agent can call. Ten steps, one real project.



Swipe to build it →

FastMCP | Streamable HTTP | Docker

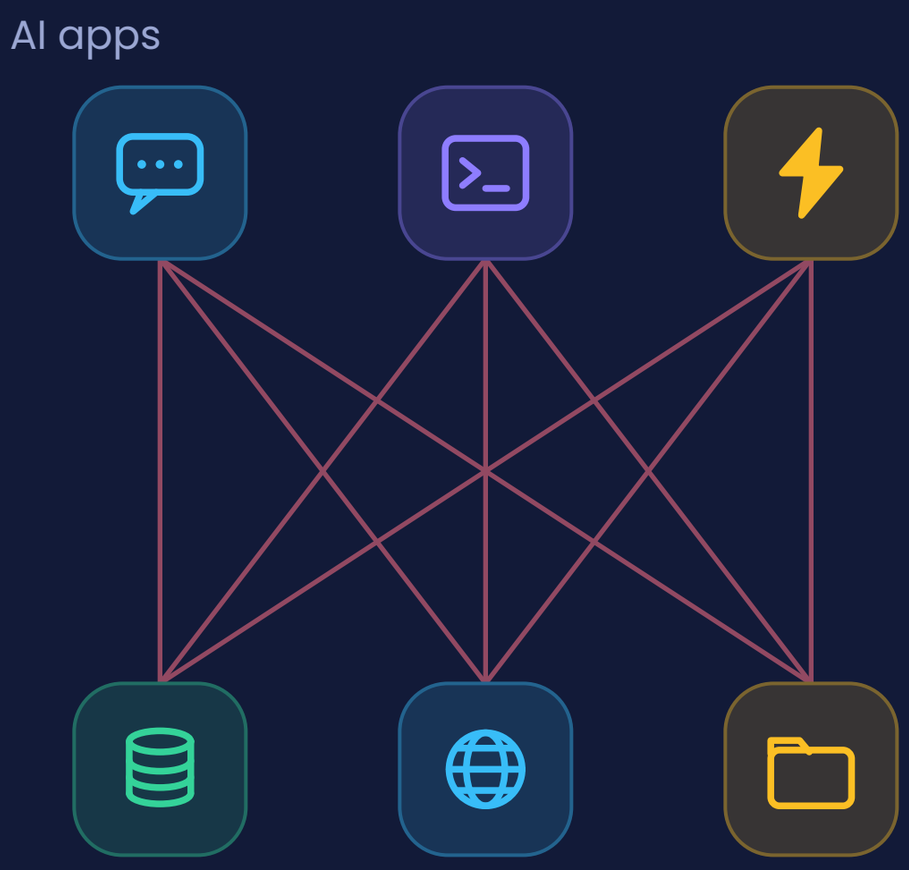
THE PROBLEM

Why MCP Exists

Every AI app wired to every tool means custom glue code, multiplied. A shared protocol turns it into addition.

BEFORE

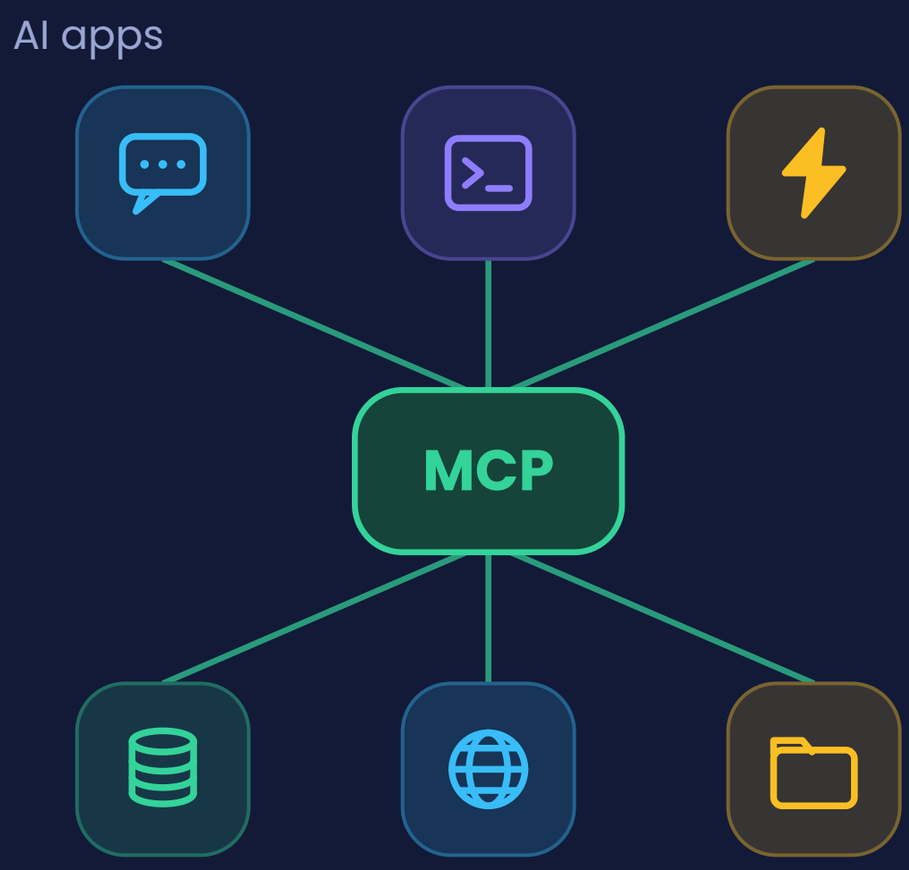
N x M integrations



9 custom connectors

AFTER

N + M with MCP



6 standard connections

Write the integration once as an MCP server. Every MCP-compatible client can use it.

CORE CONCEPTS

Three Building Blocks

A server can offer any mix of these. The big difference is who decides when each one gets used.

Tools

MODEL-CONTROLLED

Actions the AI can decide to run. They can read, write or call APIs.

```
create_ticket(title, priority)
```

Resources

APP-CONTROLLED

Read-only data addressed by a URI. The app loads it as context.

```
tickets://open
```

Prompts

USER-CONTROLLED

Reusable templates the person picks, like a slash command.

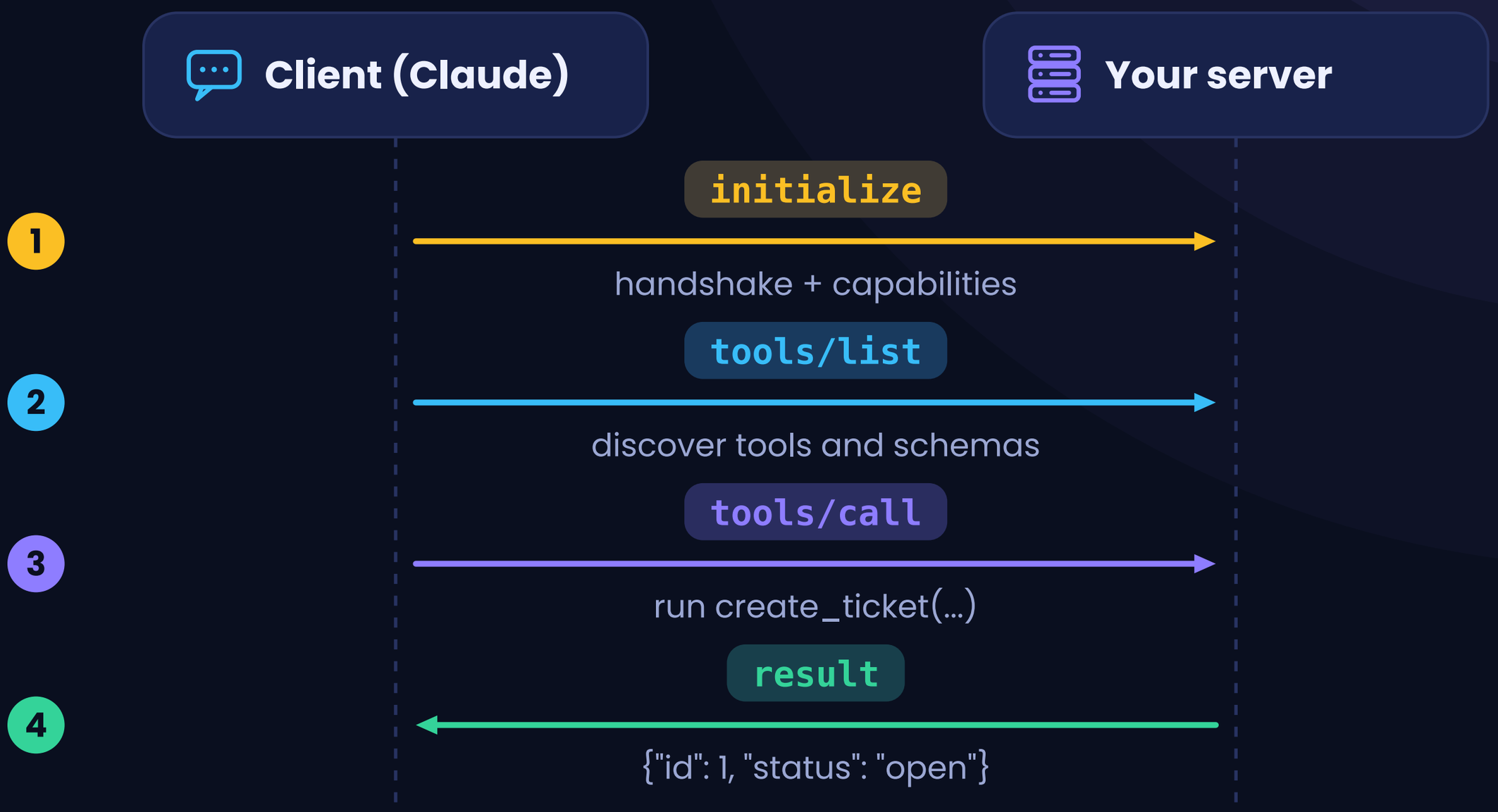
```
triage_ticket(ticket_id)
```

Our project ships 3 tools, 1 resource and 1 prompt.

UNDER THE HOOD

Anatomy of a Call

MCP is JSON-RPC 2.0. Four messages cover the whole story.



step 3: the request on the wire — JSON

```json
{
  "jsonrpc": "2.0", "id": 3,
  "method": "tools/call",
  "params": {
    "name": "create_ticket",
    "arguments": {"title": "Login bug", "priority": "high"}
  }
}
```

NETWORKING

Pick Your Transport

The transport is just how client and server exchange those messages. Choose by where the server lives.

stdio

LOCAL

Client launches your script as a subprocess.

```
Claude Desktop  ->  stdin/stdout  ->  server.py
```

Best for: development, personal tools

Streamable HTTP

RECOMMENDED

One /mcp endpoint over plain HTTP, streaming when needed.

```
Agent  ->  POST https://host/mcp  ->  server
```

Best for: production, teams, cloud

SSE

LEGACY

Older two-endpoint HTTP transport, deprecated in the spec.

```
Agent  ->  GET /sse + POST /messages
```

Only for clients that cannot do HTTP yet

Plan: develop on stdio, ship on Streamable HTTP.

STEP 1

Set Up the Project

Our example is Ticket Desk: an AI-callable support queue backed by SQLite. No vector DB, so the MCP parts stay in focus.

terminal — BASH

```
# create the project with uv
uv init ticket-desk && cd ticket-desk
uv add fastmcp

# sanity check
uv run fastmcp version
```

Prefer pip? python -m venv .venv then pip install fastmcp

Project layout

structure — LAYOUT

```
ticket-desk/
├── db.py            # SQLite data layer
├── server.py        # tools, resource, prompt
├── app.py           # HTTP app + auth
├── Dockerfile       # ship it anywhere
├── .env             # API_TOKEN (never commit)
└── pyproject.toml
```

STEP 2

Build the Data Layer

Keep storage in its own module. The MCP server will only call these helpers.

db.py

PYTHON

```python
# db.py
import os, sqlite3

DB_PATH = os.getenv("DB_PATH", "tickets.db")

def conn():
    c = sqlite3.connect(DB_PATH)
    c.row_factory = sqlite3.Row
    return c

def init():
    with conn() as c:
        c.execute("""
            CREATE TABLE IF NOT EXISTS tickets (
              id INTEGER PRIMARY KEY AUTOINCREMENT,
              title TEXT NOT NULL,
              priority TEXT DEFAULT 'medium',
              status TEXT DEFAULT 'open'
            )""")
```

STEP 3

Create Your Server

Decorate a function and it becomes a tool. Type hints and the docstring are what the model sees.

server.py PYTHON

```python
# server.py
from fastmcp import FastMCP
from fastmcp.exceptions import ToolError
import db

mcp = FastMCP(
    name="Ticket Desk",
    instructions="Manage support tickets. Search before creating."
)
db.init()

@mcp.tool
def create_ticket(title: str, priority: str = "medium") -> dict:
    """Open a ticket. priority: low, medium or high."""
    if priority not in ("low", "medium", "high"):
        raise ToolError("priority must be low, medium or high")
    with db.conn() as c:
        cur = c.execute(
            "INSERT INTO tickets(title, priority) VALUES (?, ?)",
            (title, priority))
    return {"id": cur.lastrowid, "status": "open"}
```

Always use ? placeholders. Never build SQL from model-supplied text.

STEP 4

Add More Tools

Annotations tell clients which tools only read and which change data, so they can ask for confirmation wisely.

server.py (continued) PYTHON

```python
@mcp.tool(annotations={"readOnlyHint": True})
def search_tickets(keyword: str, limit: int = 5) -> list[dict]:
    """Find tickets whose title contains the keyword."""
    with db.conn() as c:
        rows = c.execute(
            "SELECT * FROM tickets WHERE title LIKE ? LIMIT ?",
            (f"%{keyword}%", min(limit, 20))).fetchall()
    return [dict(r) for r in rows]

@mcp.tool(annotations={"idempotentHint": True})
def close_ticket(ticket_id: int) -> str:
    """Mark a ticket as closed."""
    with db.conn() as c:
        n = c.execute(
            "UPDATE tickets SET status='closed' WHERE id=?",
            (ticket_id,)).rowcount
    if n == 0:
        raise ToolError(f"Ticket {ticket_id} not found")
    return f"Ticket {ticket_id} closed"
```

ToolError messages reach the model, so it can retry or explain the problem.

STEP 5

Resource and Prompt

Beyond tools: expose a live view of open tickets, plus a reusable triage template.

server.py (continued) PYTHON

```python
@mcp.resource("tickets://open")
def open_tickets() -> str:
    """Every ticket that is still open."""
    with db.conn() as c:
        rows = c.execute(
            "SELECT id, priority, title FROM tickets "
            "WHERE status='open' ORDER BY id").fetchall()
    lines = [f"#{r['id']} [{r['priority']}] {r['title']}"
             for r in rows]
    return "\n".join(lines)

@mcp.prompt
def triage_ticket(ticket_id: int) -> str:
    """Ask the model to triage one ticket."""
    return (f"Look up ticket {ticket_id} with search_tickets, "
            "rate urgency from 1 to 5, then suggest next steps.")

if __name__ == "__main__":
    mcp.run()  # stdio by default
```

Tools act, resources inform, prompts guide. Pick the right one per job.

STEP 6

Test Before You Ship

Open the visual Inspector, then lock behaviour in with a tiny script.

terminal — BASH

```
# visual inspector in your browser (needs Node.js)
uv run fastmcp dev server.py
```

test_server.py — PYTHON

```python
import asyncio
from fastmcp import Client
from server import mcp

async def main():
    async with Client(mcp) as client:
        tools = await client.list_tools()
        print([t.name for t in tools])
        res = await client.call_tool(
            "create_ticket", {"title": "Login bug", "priority": "high"})
        print(res.data)

asyncio.run(main())
```

expected output — OUTPUT

```
['create_ticket', 'search_tickets', 'close_ticket']
{'id': 1, 'status': 'open'}
```

STEP 7

Connect Claude Desktop

Settings, Developer, Edit Config. Add the server, then fully quit and reopen the app.

claude_desktop_config.json — JSON

```
{
  "mcpServers": {
    "ticket-desk": {
      "command": "uv",
      "args": ["--directory", "/abs/path/ticket-desk",
               "run", "server.py"],
      "env": { "DB_PATH": "/abs/path/ticket-desk/tickets.db" }
    }
  }
}
```

where the file lives — PATHS

```
macOS    ~/Library/Application Support/Claude/
Windows  %APPDATA%\Claude\
```

⚠ stdio servers must never print() to stdout. It corrupts the stream. Log to stderr.

STEP 8

Go Remote with Auth

Same tools over Streamable HTTP, behind a bearer token.

app.py — PYTHON

```python
import os, hmac
from starlette.middleware import Middleware
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.responses import JSONResponse
from server import mcp

TOKEN = os.environ["API_TOKEN"]

class BearerAuth(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        sent = request.headers.get("authorization", "")
        if not hmac.compare_digest(sent, f"Bearer {TOKEN}"):
            return JSONResponse({"error": "unauthorized"}, 401)
        return await call_next(request)

app = mcp.http_app(middleware=[Middleware(BearerAuth)])
```

terminal — BASH

```bash
API_TOKEN=(openssl rand -hex 32) uvicorn app:app --port 8000
# MCP endpoint -> http://localhost:8000/mcp
```

Fine for a private server. For many users, move to OAuth.

STEP 9

Dockerize the Server

One image runs identically on your laptop, a VPS or a managed platform.

Dockerfile — DOCKERFILE

```dockerfile
FROM python:3.12-slim
COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/uv
WORKDIR /app
COPY pyproject.toml uv.lock* ./
RUN uv sync --no-dev
COPY . .
ENV PATH="/app/.venv/bin:$PATH" DB_PATH=/data/tickets.db
VOLUME /data
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

terminal — BASH

```bash
docker build -t ticket-desk .

docker run -d -p 8000:8000 \
  -e API_TOKEN=change-me \
  -v ticket_data:/data \
  ticket-desk
```

The volume keeps tickets.db alive across container restarts.

STEP 10

Deploy to the Cloud

Fly.io example. Railway, Render or any VPS works the same way.

terminal — BASH

```bash
fly launch --no-deploy          # detects the Dockerfile
fly volumes create data --size 1
fly secrets set API_TOKEN=(openssl rand -hex 32)
fly deploy
```

fly.toml (the parts you edit) — TOML

```toml
[http_service]
  internal_port = 8000
  force_https = true

[mounts]
  source = "data"
  destination = "/data"
```

smoke test — BASH

```bash
curl -i https://ticket-desk.fly.dev/mcp
# HTTP/2 401   <- no token, correctly refused
```

Your server now lives at https://ticket-desk.fly.dev/mcp

STEP 11

Connect the Remote Server

Same tools, now served from the cloud.

Claude Code — BASH

```
claude mcp add --transport http ticket-desk \
  https://ticket-desk.fly.dev/mcp \
  --header "Authorization: Bearer $API_TOKEN"
```

Claude Desktop (via mcp-remote bridge) — JSON

```
"ticket-desk-remote": {
  "command": "npx",
  "args": ["mcp-remote", "https://ticket-desk.fly.dev/mcp",
           "--header", "Authorization: Bearer ${API_TOKEN}"],
  "env": { "API_TOKEN": "your-token" }
}
```

claude.ai custom connectors sign in with OAuth, so add OAuth before sharing with a team.

HARDENING

Production Checklist

A tool is a doorway into your systems. Treat every call as untrusted input.

Authenticate

Rotate tokens. Use OAuth for real users.

Validate inputs

Check types, ranges and allow-lists. Parameterize SQL.

Least privilege

Read-only DB users. Mark read-only tools with hints.

Log every call

Tool name, duration and outcome. Never log secrets.

Limit usage

Timeouts, result size caps and per-token rate limits.

Distrust content

Text in tickets or docs can carry hidden instructions.

TROUBLESHOOTING

Debugging Cheatsheet

Symptom first, then the usual suspects in order.

Server missing in Claude

- → Config paths are absolute
- → Valid JSON, then full restart
- → Run fastmcp dev server.py
- → Check Claude's MCP logs

Tool call fails or hangs

- → Reproduce it in the Inspector
- → Read the ToolError text
- → No print() on stdio servers
- → API_TOKEN set in env block?

Remote server unreachable

- → URL ends with /mcp
- → Token matches header exactly
- → Port and firewall are open
- → Run docker logs / fly logs

RECAP

What You Shipped

A real, callable, secured MCP server. Here is what is next.

SHIPPED

- 3 tools, 1 resource, 1 prompt
- SQLite data layer
- Local server over stdio
- Remote Streamable HTTP
- Bearer token auth
- Docker image
- Cloud deployment

NEXT UP

- → OAuth for team access
- → Postgres instead of SQLite
- → Structured logging + metrics
- → Automated test suite in CI
- → Compose several servers
- → Rate limits and quotas

Follow @Zakaurrehman

More hands-on AI engineering guides. Repost to help a builder.